

Lesson 5: Git Interactive Rebase, Stashing Tricks & Advanced Clean-up

In Lesson 4, you mastered basic safety nets like `git restore`, `git revert`, and simple stashing. In this final advanced lesson, you will learn how to rewrite and polish your local commit history before opening a Pull Request, manage complex stashes, and keep your local repository light and clean.

1. Cleaning History with Interactive Rebase (`git rebase -i`)

Before sharing your code with your team via a Pull Request, your local branch might contain messy commits like "fix typo", "wip", or "test fix". Interactive rebasing lets you clean, combine, and rename commits so your history looks clean and intentional.

Bash

Interactively rebase the last 3 commits on your current branch

```
git rebase -i HEAD~3
```

When you run this command, Git opens an editor listing your commits with action commands:

Plaintext

pick a1b2c3d Add initial user profile component

pick e4f5g6h Fix typo in CSS

pick i7j8k9l Add responsive styles to profile

Rebase Commands:

p, pick = use commit

r, reword = use commit, but edit the commit message

s, squash = meld commit into previous commit (combines them)

drop = remove commit

Common Clean-up Tasks:

- Squashing: Change pick to squash (or s) on smaller fixup commits to merge them into the parent commit above them.
- Rewording: Change pick to reword (or r) to fix a bad or vague commit message.
- Dropping: Change pick to drop (or just delete the line) to completely remove an unwanted commit.

⚠ Golden Rule of Rebasing: Never rebase commits that have already been pushed to a public or shared branch (like main). Only rebase your local feature branches!

2. Advanced Stashing Techniques

Beyond basic `git stash` and `git stash pop`, Git lets you maintain multiple stashed states and inspect their contents:

Bash

Stash work with a descriptive name

```
git stash save "WIP: Header component alignment"
```

List all saved stashes in your stack

```
git stash list
```

Output:

stash@{0}: On feature-ui: WIP: Header component alignment

stash@{1}: On main: Temporary save for hotfix

Inspect what changed inside a specific stash without applying it

```
git stash show -p stash@{0}
```

Apply a specific stash and keep it in the stash list

```
git stash apply stash@{0}
```

Clear all stashes from your stack

```
git stash clear
```

3. Cleaning Untracked Files (git clean)

Over time, build processes, dependency installers, or local tests create untracked temporary files and folders. `git clean` removes these untracked files quickly.

Bash

Always do a dry run first to see what WILL be deleted

```
git clean -nd
```

Delete all untracked files forcibly

```
git clean -fd
```

Delete untracked files AND ignored files (like node_modules or build artifacts)

```
git clean -xfd
```

4. Recovering "Lost" Commits with git reflog

If you ever accidentally delete a branch, misplace a commit during a rebase, or hard reset to the wrong commit, Git rarely deletes data immediately. git reflog records every move of your HEAD pointer locally.

Bash

View the master history of all local HEAD movements

```
git reflog
```

How to recover a deleted state:

1. Find the commit hash associated with the state you lost (e.g., HEAD@{3} or c1d2e3f).
2. Restore your workspace to that point or create a branch off it:

Bash

```
git checkout -b recovered-work c1d2e3f
```

Practical Exercise

Master local history cleanup and recovery:

1. Make two quick edits in notes.txt. Commit the first as "add feature part 1" and the second as "typo fix".
2. Run git log --oneline to view both commits.
3. Run git rebase -i HEAD~2 and change the word pick on the second commit to squash. Save and close the editor.
4. Update the combined commit message to "Add complete feature" and save. Run git log --oneline to verify the two messy commits are now one clean commit!
5. Run git reflog to inspect how Git tracked your actions behind the scenes.